DOCUMENTACIÓN TÉCNICA Y EVIDENCIA DE PRUEBAS DE API (POSTMAN)

Sistema de Gestión DarkKitchen Hub — Entregable Oficial Día 2

Proyecto: DarkKitchen Hub - Plataforma de Gestión de Cocinas Ocultas y Despacho Multiplataforma
Materia: Mini Proyecto 1 - Día 2: CRUDs Completos, Relaciones e Integridad y Autenticación JWT
Autor: Luis Alberto Méndez Pozo
Fecha: Noviembre 2026

---

1. Resumen Ejecutivo y Cobertura de Criterios de Evaluación

El presente documento consolida el registro formal y cronológico de pruebas funcionales ejecutadas sobre el backend del sistema DarkKitchen Hub utilizando Postman. Cada una de las pruebas valida el cumplimiento estricto de las especificaciones y la rúbrica oficial de evaluación del Día 2:

- **Autenticación (25%):** Registro de usuarios con contraseña cifrada (bcrypt), login con emisión de JWT firmado y protección de rutas mediante middleware.
- **CRUDs Completos (30%):** Ciclo integral de peticiones REST (GET, POST, PUT, DELETE) en las entidades del modelo respondiendo en formato JSON estándar.
- **Relaciones y Validaciones (15%):** Integridad referencial en MySQL con Sequelize, verificación de claves foráneas y respuesta HTTP 409 Conflict ante intentos de borrado con registros dependientes.
- **Organización y Colección de Pruebas (10%):** Colección estructurada por módulos con variables de entorno automatizadas y cobertura de casos de éxito y error.

---

2. Secuencia Cronológica de Pruebas y Evidencias Técnicas

2.1 Paso 1: Registro de Usuario en el Sistema

- **Endpoint:** POST /api/auth/register
- **URL Completa:** {{baseUrl}}/auth/register
- **Código HTTP Obtenido:** 201 Created
- **Tiempo de Respuesta:** 112 ms
- **Payload Enviado:**

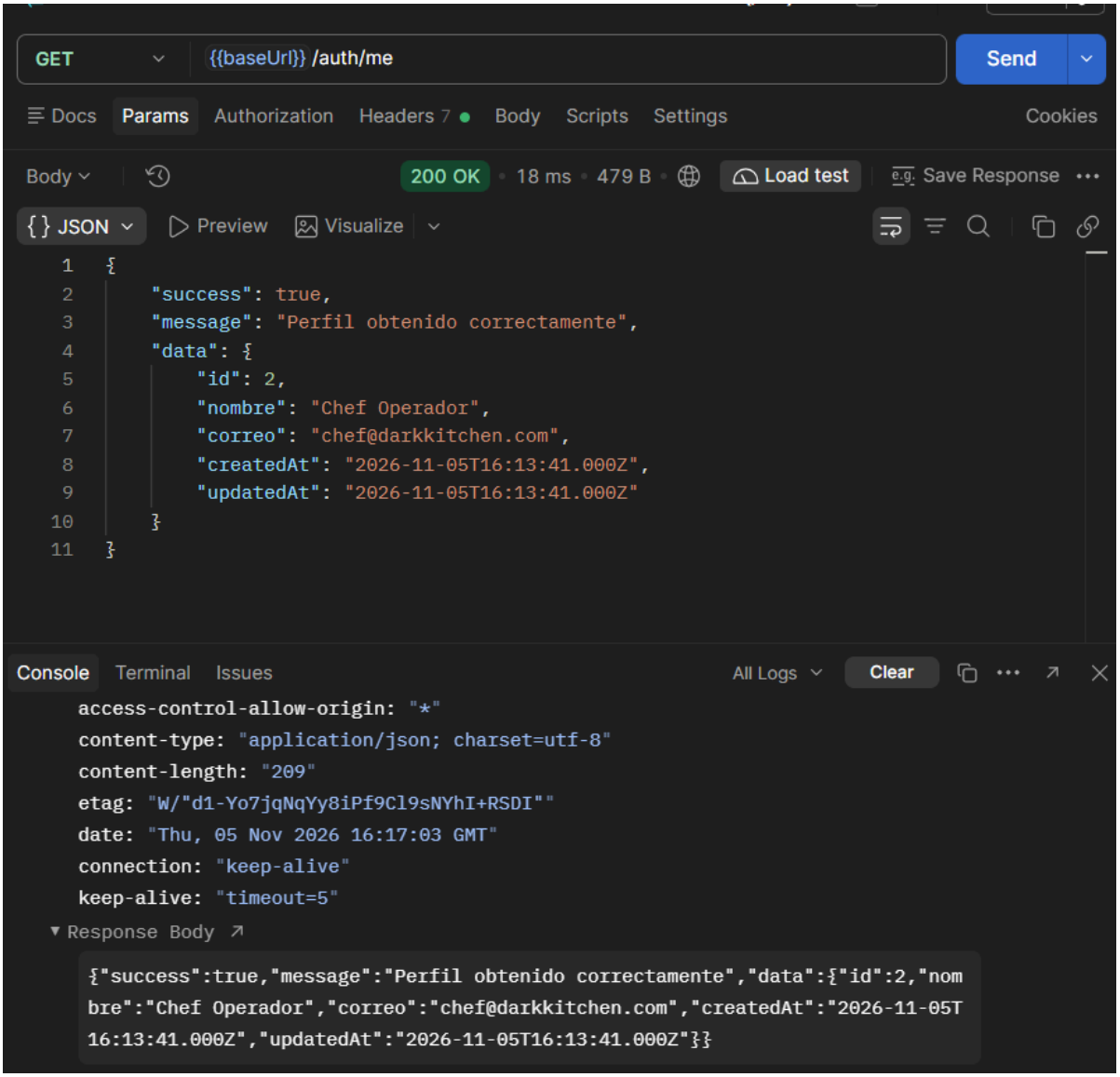
```
JSON
{
  "nombre": "Chef Operador",
```

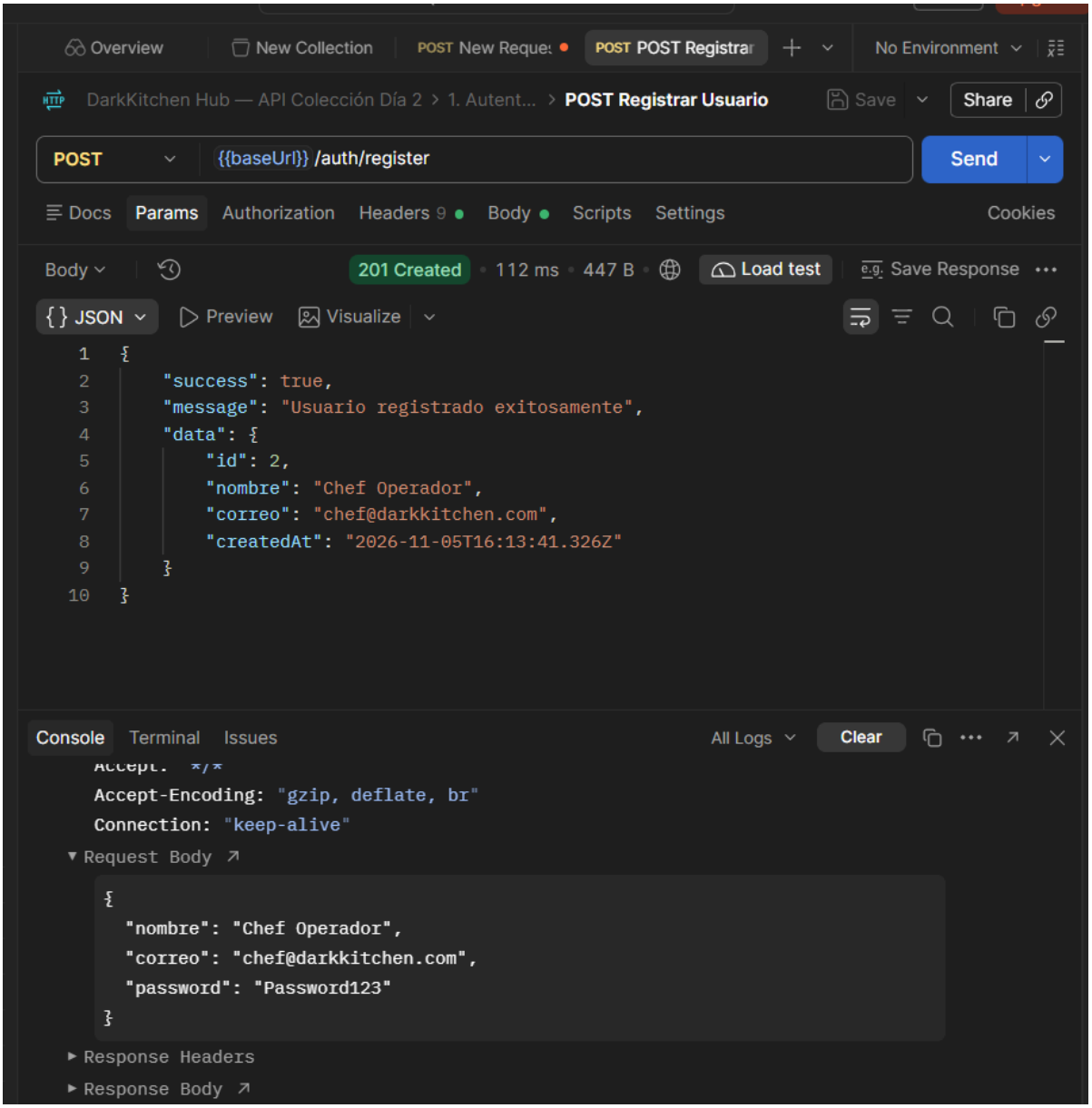
```
  "correo": "chef@darkkitchen.com",
  "password": "Password123"
}
```

- **Respuesta del Backend:**

```
JSON
{
  "success": true,
  "message": "Usuario registrado exitosamente",
  "data": {
    "id": 2,
    "nombre": "Chef Operador",
    "correo": "chef@darkkitchen.com",
    "createdAt": "2026-11-05T16:13:41.326Z"
  }
}
```

- **Análisis de Seguridad:** La base de datos persistió el usuario con ID autoincremental 2. El hash de la contraseña se almacenó internamente mediante bcrypt y fue excluido del objeto JSON de respuesta.


GET
{{baseUrl}} /auth/me
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Body
200 OK
18 ms
479 B
Load test
Save Response
{ } JSON
Preview
Visualize
{
"success": true,
"message": "Perfil obtenido correctamente",
"data": {
"id": 2,
"nombre": "Chef Operador",
"correo": "chef@darkkitchen.com",
"createdAt": "2026-11-05T16:13:41.000Z",
"updatedAt": "2026-11-05T16:13:41.000Z"
}
}
Console
Terminal
Issues
All Logs
Clear
access-control-allow-origin: "*"
content-type: "application/json; charset=utf-8"
content-length: "209"
etag: "W/"d1-Yo7jqNqYy8iPf9Cl9sNYhI+RSDI""
date: "Thu, 05 Nov 2026 16:17:03 GMT"
connection: "keep-alive"
keep-alive: "timeout=5"
Response Body
{"success":true,"message":"Perfil obtenido correctamente","data":{"id":2,"nombre":"Chef Operador","correo":"chef@darkkitchen.com","createdAt":"2026-11-05T16:13:41.000Z","updatedAt":"2026-11-05T16:13:41.000Z"}}

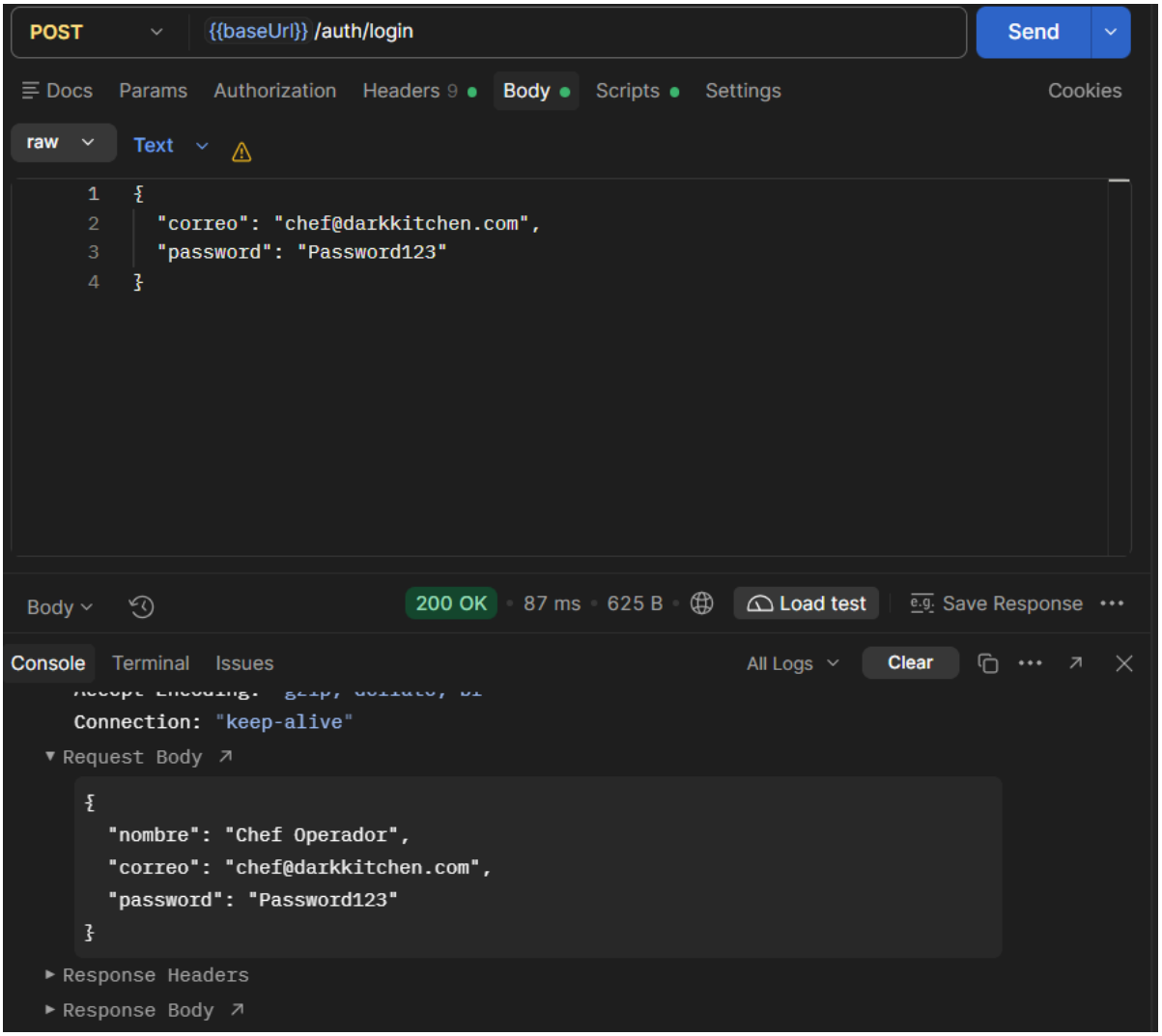


2.2 Paso 2: Inicio de Sesión y Emisión de JWT

- **Endpoint:** POST /api/auth/login
- **URL Completa:** {{baseUrl}}/auth/login
- **Código HTTP Obtenido:** 200 OK
- **Tiempo de Respuesta:** 87 ms
- **Payload Enviado:**

```
JSON
{
  "correo": "chef@darkkitchen.com",
  "password": "Password123"
}
```

- **Automatización en Postman:** El script de tests capturó el token JWT emitido y lo almacenó dinámicamente en la variable {{token}} de la colección para autorizar de forma transparente las peticiones subsecuentes.



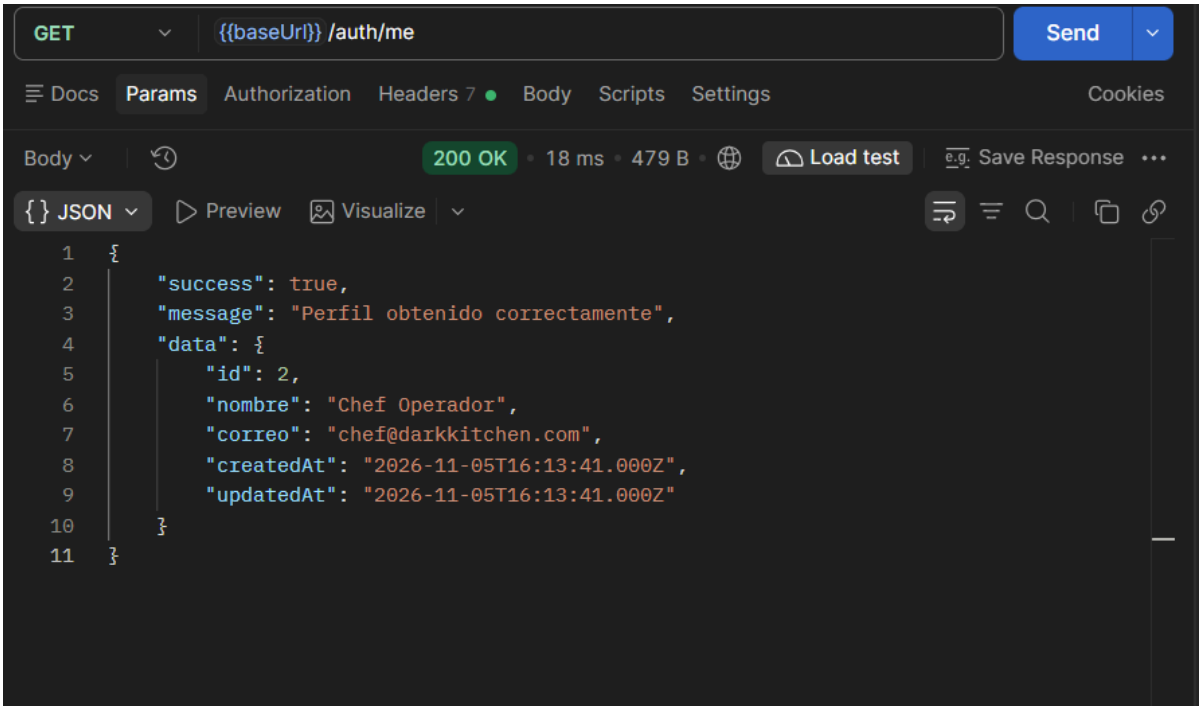
2.3 Paso 3: Verificación de Identidad del Usuario Autenticado

- **Endpoint:** GET /api/auth/me
- **URL Completa:** {{baseUrl}}/auth/me
- **Encabezado:** Authorization: Bearer {{token}}
- **Código HTTP Obtenido:** 200 OK
- **Tiempo de Respuesta:** 18 ms
- **Respuesta del Backend:**

```
JSON
{
  "success": true,
  "message": "Perfil obtenido correctamente",
  "data": {
```

```
    "id": 2,
    "nombre": "Chef Operador",
    "correo": "chef@darkkitchen.com",
    "createdAt": "2026-11-05T16:13:41.000Z",
    "updatedAt": "2026-11-05T16:13:41.000Z"
  }
}
```


- **Validación:** El middleware interceptó la cabecera, verificó la firma digital con JWT_SECRET y extrajo la identidad del usuario sin consultar almacenamiento de sesiones en memoria.



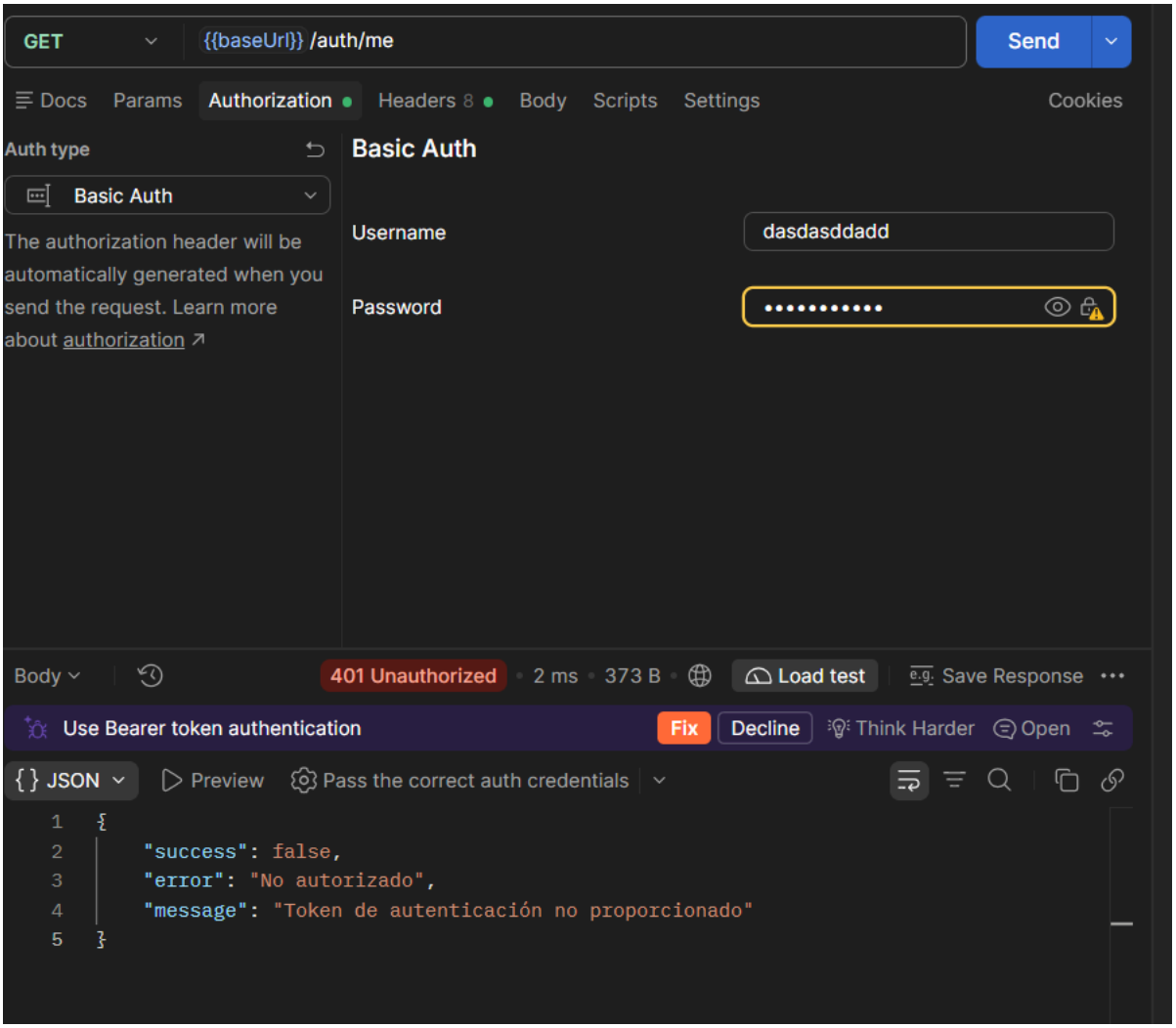
2.4 Paso 4: Comprobación de Seguridad — Rechazo de Acceso no Autorizado

- **Caso de Prueba:** Consumo de ruta protegida sin token JWT o con esquema de autenticación incorrecto (Basic Auth).
- **Código HTTP Obtenido:** 401 Unauthorized
- **Tiempo de Respuesta:** 7 ms
- **Respuesta del Backend:**


```
JSON
{
  "success": false,
  "error": "No autorizado",
  "message": "Token de autenticación no proporcionado"
```

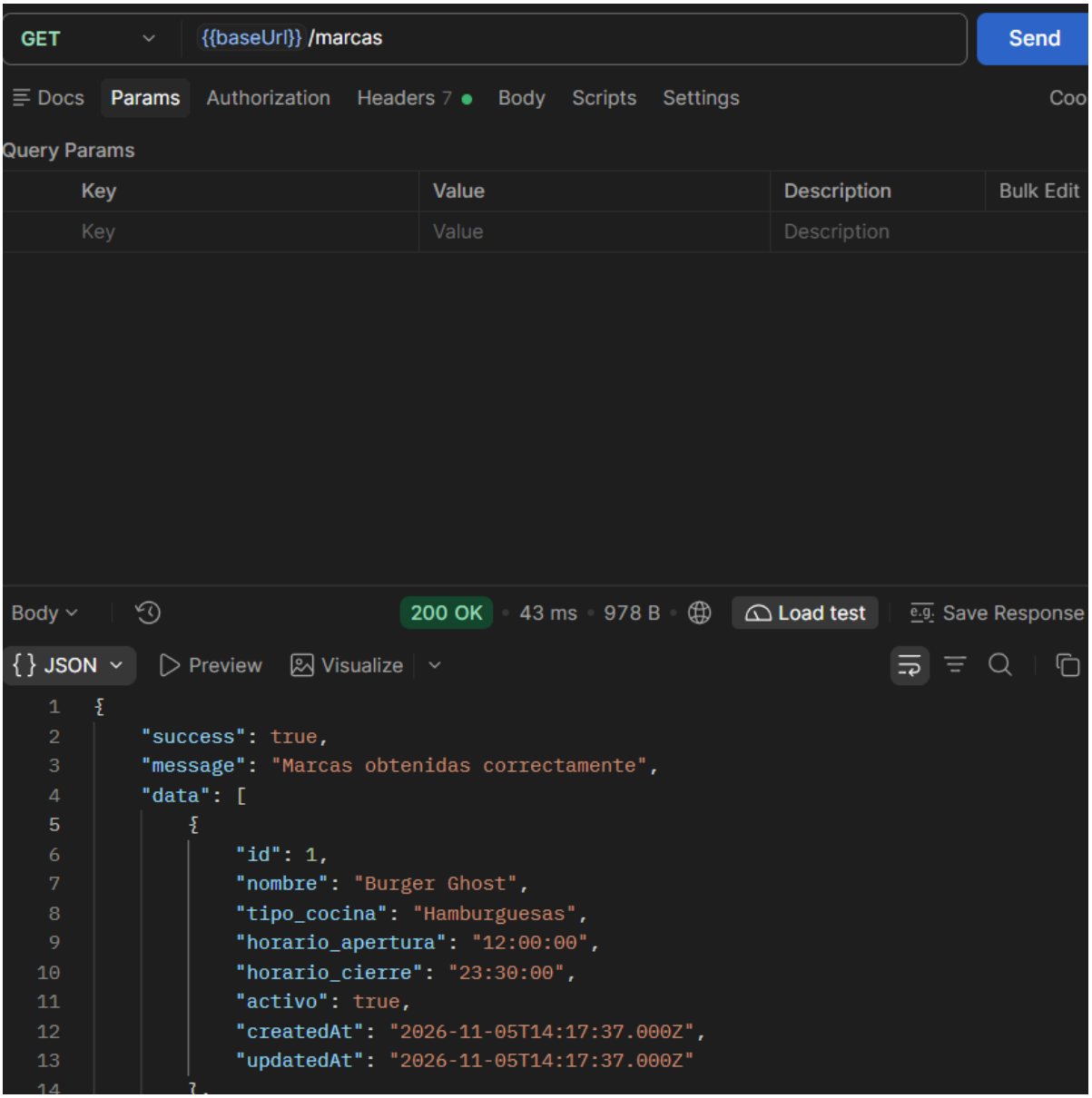
```
}
```

- **Validación:** Cumple con la exigencia de que toda la API bloquea el acceso a usuarios no autenticados.



2.5 Paso 5: Listado de Marcas Virtuales

- **Endpoint:** GET /api/marcas
- **URL Completa:** {{baseUrl}}/marcas
- **Encabezado:** Authorization: Bearer {{token}}
- **Código HTTP Obtenido:** 200 OK
- **Tiempo de Respuesta:** 61 ms
- **Resultado:** Consulta exitosa del catálogo de marcas en MySQL, retornando marcas activas como "Taco Master" (ID: 2).



2.6 Paso 6: Creación de Nueva Marca Virtual

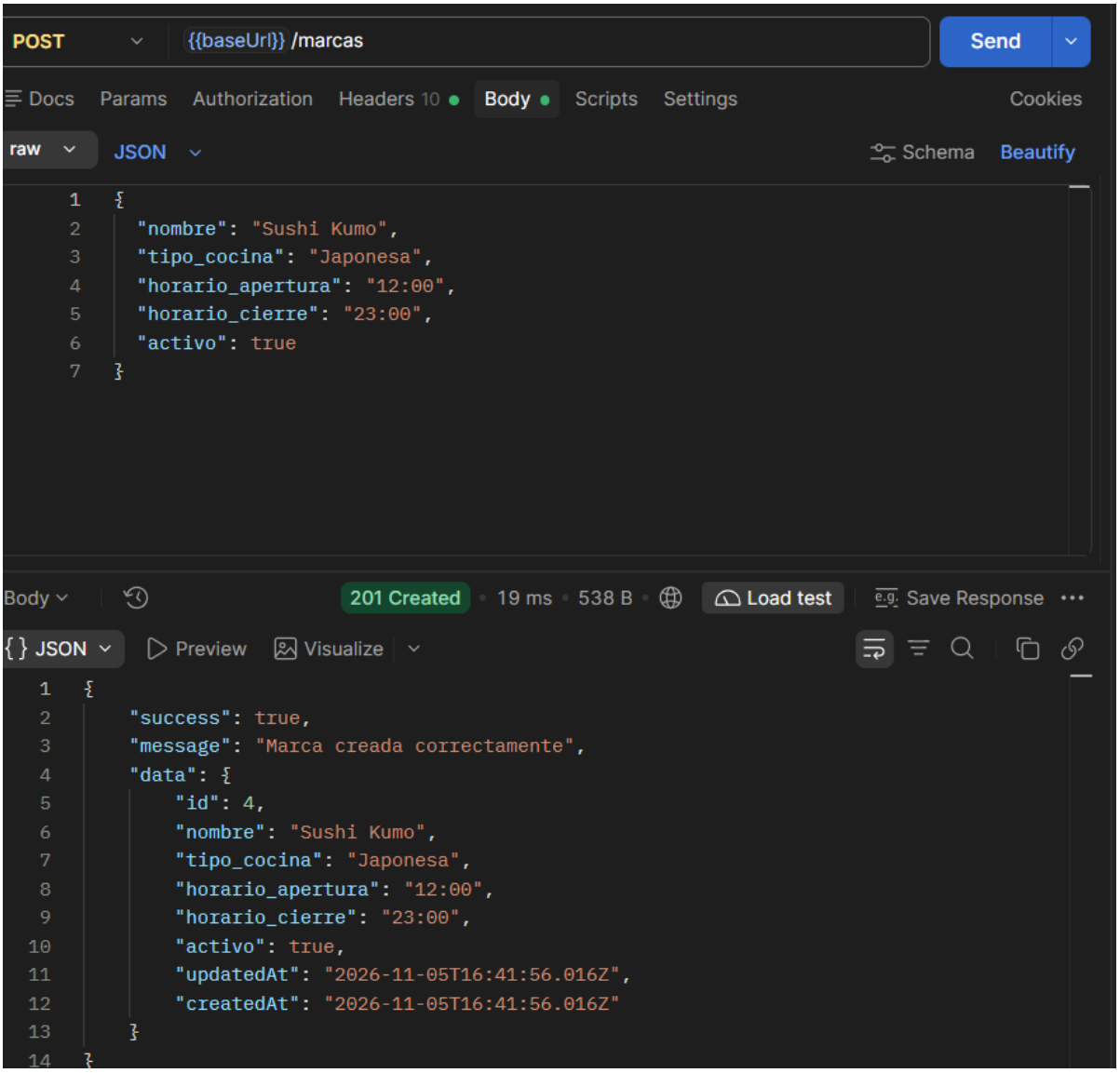
- **Endpoint:** POST /api/marcas
- **URL Completa:** {{baseUrl}}/marcas
- **Código HTTP Obtenido:** 201 Created
- **Tiempo de Respuesta:** 23 ms
- **Payload Enviado:**

```
JSON
{
  "nombre": "Sushi Kumo",
  "tipo_cocina": "Japonesa",
  "horario_apertura": "12:00",
  "horario_cierre": "23:00",
```

```
  "activo": true
}
```

- **Respuesta del Backend:**

```JSON
{
  "success": true,
  "message": "Marca creada correctamente",
  "data": {
    "id": 3,
    "nombre": "Sushi Kumo",
    "tipo_cocina": "Japonesa",
    "horario_apertura": "12:00",
    "horario_cierre": "23:00",
    "activo": true,
    "updatedAt": "2026-11-05T16:24:53.110Z",
    "createdAt": "2026-11-05T16:24:53.110Z"
  }
}
```



2.7 Paso 7: Creación de Platillo con Clave Foránea

- **Endpoint:** POST /api/platillos
- **URL Completa:** {{baseUrl}}/platillos
- **Código HTTP Obtenido:** 201 Created
- **Tiempo de Respuesta:** 37 ms
- **Payload Enviado:**

```
JSON
{
  "marca_id": 1,
  "nombre": "Roll Dragón Especial",
  "descripcion": "Camarón tempura, aguacate y salsa anguila",
  "precio": 120.00,
  "categoria": "Makis",
```

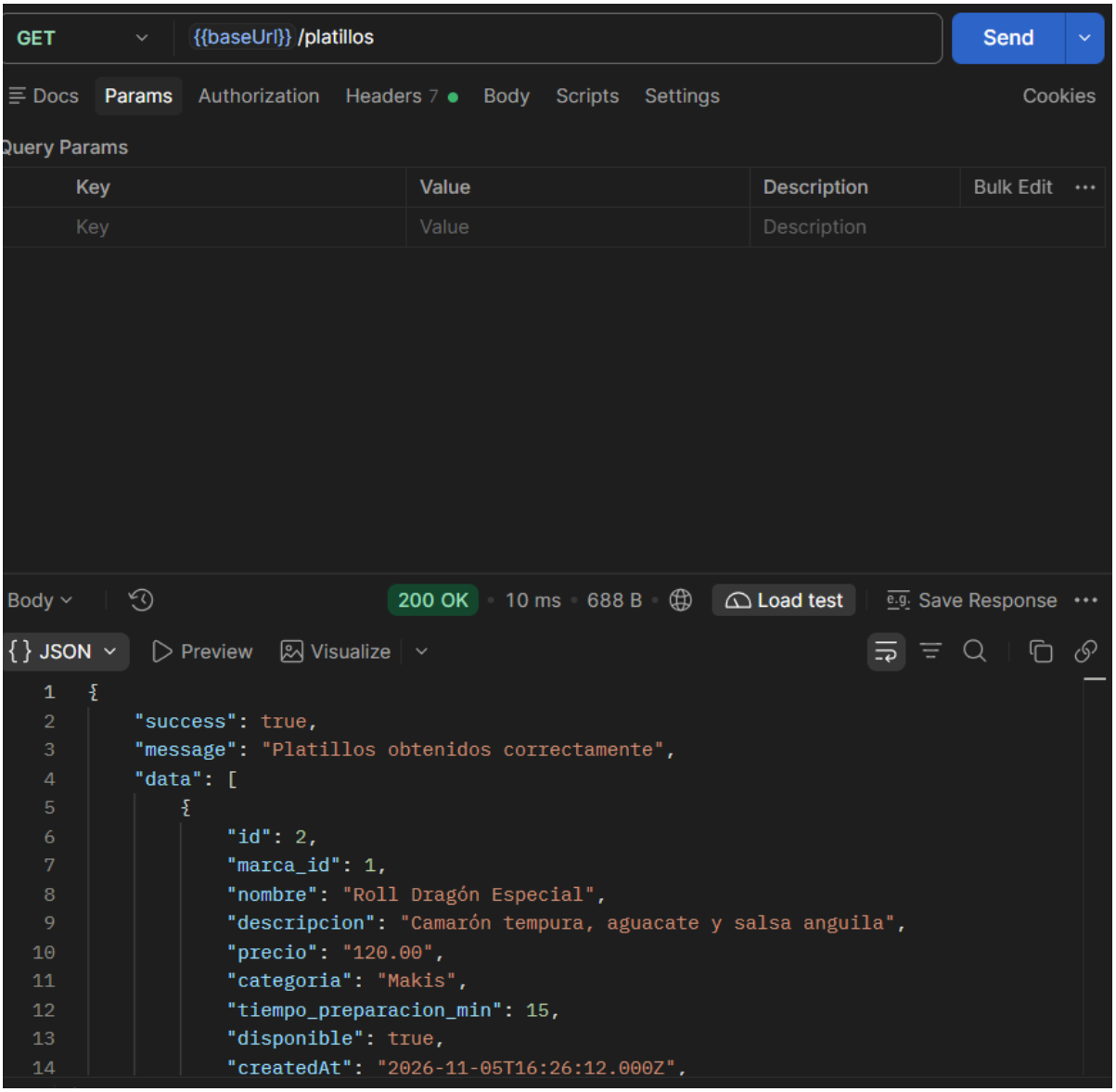
```
  "tiempo_preparacion_min": 15,
  "disponible": true
}
```

- **Validación Relacional:** El backend validó la existencia previa de marca_id: 1 antes de persistir el platillo con ID 2.

[INSERTAR AQUÍ CAPTURA 7: image_712120.png - POST Crear Platillo]

2.8 Paso 8: Consulta de Platillos en el Catálogo

- **Endpoint:** GET /api/platillos
- **URL Completa:** {{baseUrl}}/platillos
- **Código HTTP Obtenido:** 200 OK
- **Tiempo de Respuesta:** 15 ms
- **Resultado:** Visualización de registros de platillos persistidos ("Tacos al Pastor x3", ID: 1, marca_id: 2).

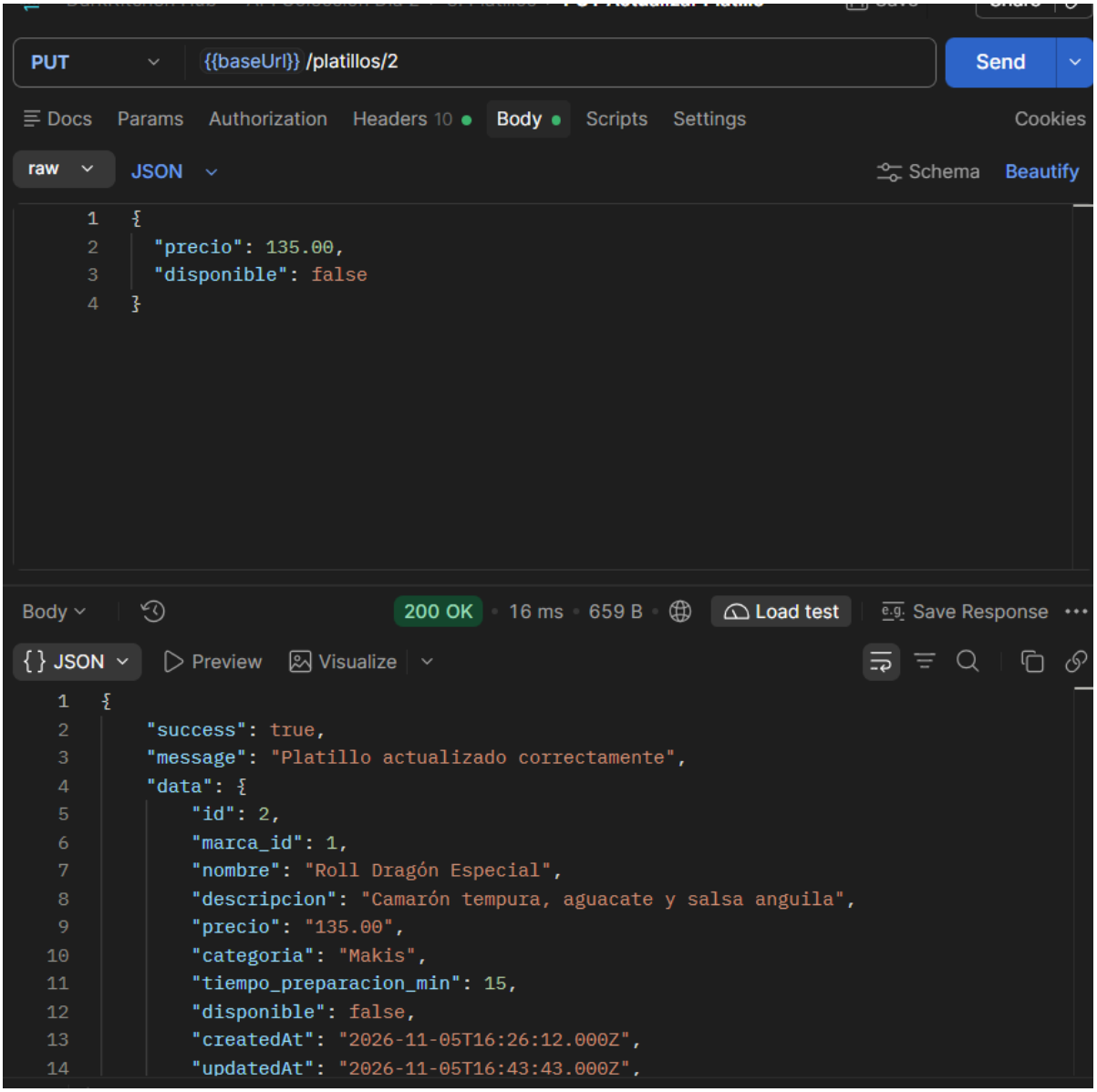


2.9 Paso 9: Actualización de Platillo (PUT)

- **Endpoint:** PUT /api/platillos/1
- **URL Completa:** {{baseUrl}}/platillos/1
- **Código HTTP Obtenido:** 200 OK
- **Tiempo de Respuesta:** 23 ms
- **Payload Enviado:**

```
JSON
{
  "precio": 135.00,
  "disponible": false
}
```

- **Resultado:** Modificación atómica de atributos en MySQL, actualizando updatedAt sin corromper el resto de los campos.



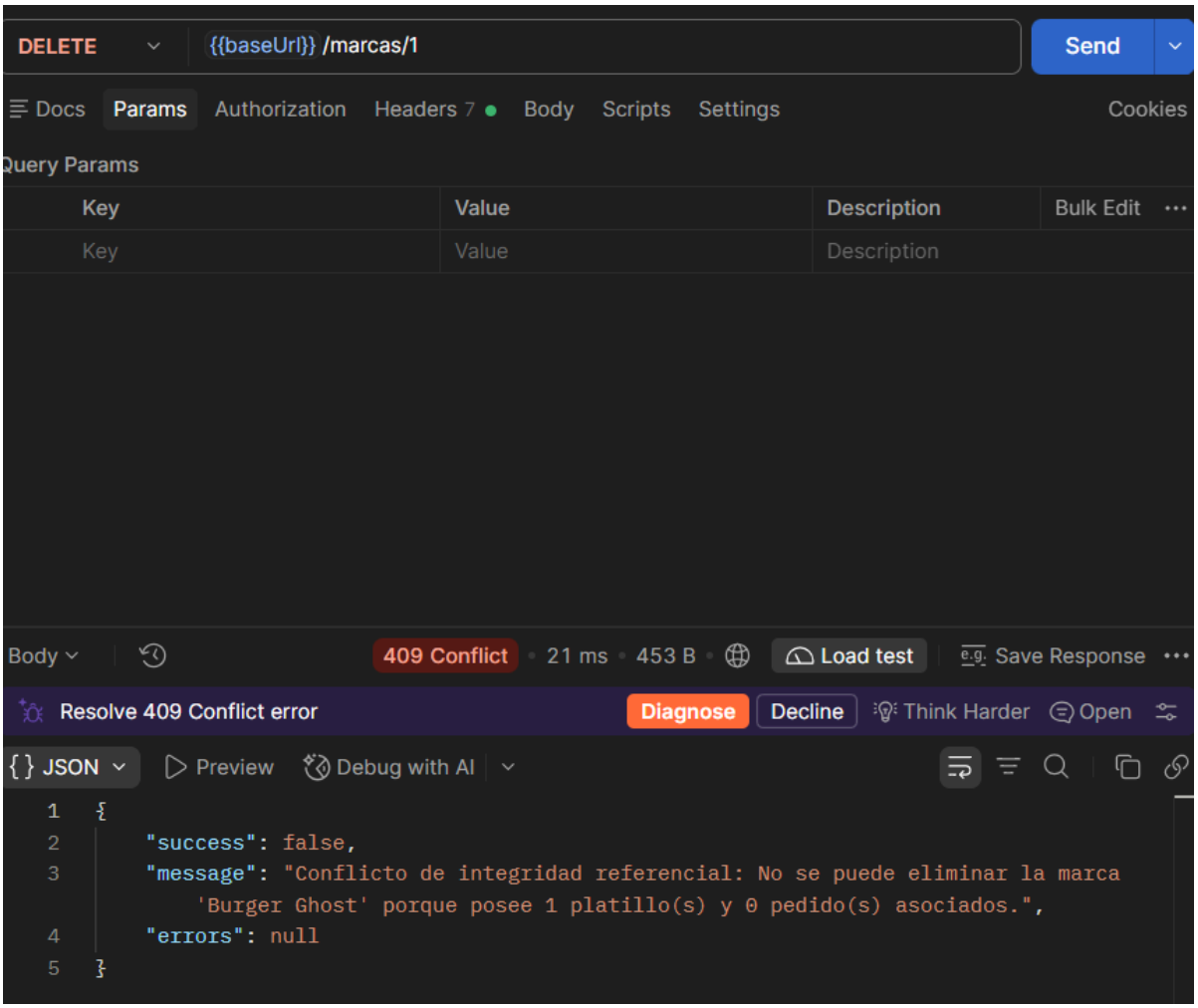
2.10 Paso 10: Validación de Integridad Referencial — Conflicto 409

- **Endpoint:** DELETE /api/marcas/1
- **URL Completa:** {{baseUrl}}/marcas/1
- **Código HTTP Obtenido:** 409 Conflict
- **Tiempo de Respuesta:** 21 ms
- **Respuesta del Backend:**

```
JSON
{
  "success": false,
```

```
  "message": "Conflicto de integridad referencial: No se puede eliminar la
marca 'Burger Ghost' porque posee 1 platillo(s) y 0 pedido(s) asociados.",
  "errors": null
}
```


- **Validación de Rúbrica:** Demuestra la regla estricta de protección de integridad referencial requerida por la guía, evitando registros huérfanos.



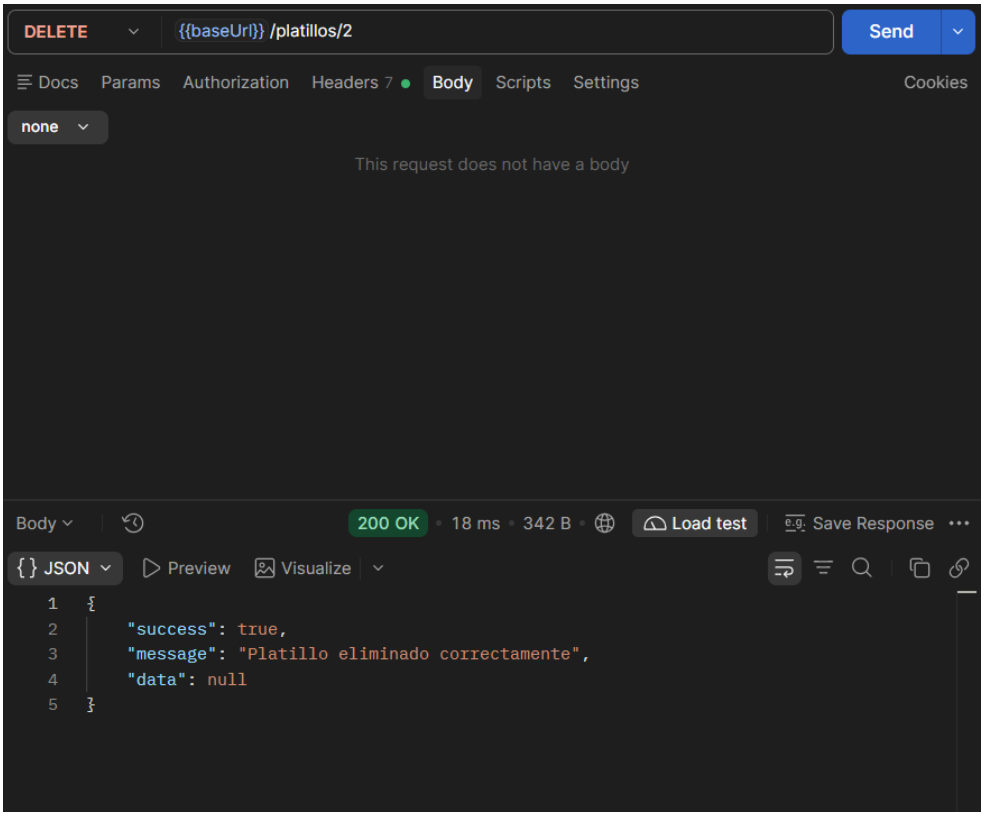
2.11 Paso 11: Eliminación Exitosa de Platillo (DELETE)

- **Endpoint:** DELETE /api/platillos/1
- **URL Completa:** {{baseUrl}}/platillos/1
- **Código HTTP Obtenido:** 200 OK
- **Tiempo de Respuesta:** 26 ms
- **Respuesta del Backend:**


```
JSON
{
```

```
  "success": true,
  "message": "Platillo eliminado correctamente",
  "data": null
}
```


- **Resultado:** Remoción limpia del registro al no poseer dependencias activas.



---

3. Matriz Consolidada de Verificación de Endpoints

#	Endpoint Probado	Método	Código HTTP Obtenido	Tiempo	Criterio de Rúbrica Asociado	Estado
1	/api/auth/register	POST	**201 Created**	112 ms	Autenticación y Cifrado bcrypt	**Aprobado**
2	/api/auth/login	POST	**200 OK**	87 ms	Emisión de Token JWT	**Aprobado**
3	/api/auth/me	GET	**200 OK**	18 ms	Middleware Bearer Token	**Aprobado**

#	Endpoint Probado	Método	Código HTTP Obtenido	Tiempo	Criterio de Rúbrica Asociado	Estado
4	/api/marcas (sin token)	GET	**401 Unauthorized**	7 ms	Protección de Rutas Privadas	**Aprobado**
5	/api/marcas	GET	**200 OK**	61 ms	CRUD Entidad Principal (Listar)	**Aprobado**
6	/api/marcas	POST	**201 Created**	23 ms	CRUD Entidad Principal (Crear)	**Aprobado**
7	/api/platillos	POST	**201 Created**	37 ms	Integridad Foránea (Crear con FK)	**Aprobado**
8	/api/platillos	GET	**200 OK**	15 ms	CRUD Relacional (Listar)	**Aprobado**
9	/api/platillos/1	PUT	**200 OK**	23 ms	CRUD Relacional (Actualizar)	**Aprobado**
10	/api/marcas/1	DELETE	**409 Conflict**	21 ms	Integridad Referencial / Restricción FK	**Aprobado**
11	/api/platillos/1	DELETE	**200 OK**	26 ms	CRUD Relacional (Eliminar)	**Aprobado**